FitXchange

Project Reflection

Introduction

FitXchange is a web-based application designed to facilitate secure and eco-friendly outfit sharing. By combining a MySQL database, a Node.js/Express back end, and Pug/CSS on the front end, we aimed to let users list, borrow, and exchange outfits easily while reducing fashion waste. My role as Product Manager involved guiding overall development, ensuring design consistency, and maintaining clear alignment across team members.

Design Process

From the outset, we identified user stories like “I need an outfit for a specific event” or “I want to rent out my items.” and progressed to define our feature set, wireframes and use case diagrams to structure these stories. Drawing from existing platforms like Amazon, Shien, Burberry and Asos we adapted proven design patterns to accelerate our own feature development. One key design choice was eliminating any dedicated “outfits” class in favor of a singular Inventory entity, ensuring that all items share uniform fields and relationships. This also made queries and sorting more consistent.

Individual Perspective

For me, breaking the codebase into well-defined components (controllers, models, views) was pivotal. This compartmentalization meant I could manage intricate data relationships; like those connecting Inventory and Category. It also helped identify performance issues early, especially when implementing advanced filtering (sorting by price, category, or location).

I found referencing existing designs for quick UI decisions immensely helpful. Similar to how some sites handle listing pages and search filters, I borrowed interface patterns to expedite user-facing features. This approach ensured we maintained clarity and consistency in our design choices.

Group Perspective

As a team, we practiced agile principles by setting up frequent mini-stand-ups and employing a shared Kanban board on GitHub. This allowed us to quickly manage and redistribute tasks when other members encountered unforeseen difficulties. We also established standard design guidelines like using Pug for templating our website sitewide layout along with consistent error handling with Express middleware.

One area I would have liked to explore further is blockchain integration for tracking physical items. Because these are tangible goods changing hands, embedding a distributed ledger system could improve traceability covering every stage from listing an outfit to returning it. Such a solution would also help manage our digital footprint more transparently, reducing global waste in multiple ways by facilitating a reliable chain of custody and encouraging more responsible consumption.

FitXchange

Development

Key Contributions and Insights

- Database & Queries: I frequently reviewed schema decisions for example joining table outfit and inventory to ensure efficient data retrieval from the database. Libraries like mysql2 were crucial in connecting to our MySQL instance.

- AI/Chatbot Integration: Integrating openai with node-nlp powered our fashion advice chatbot. It addresses user queries about outfit recommendations and site policies. Despite the initial steep learning curve, this ultimately streamlined our Q&A flow and enhanced user engagement.

- Secure User Management: Password hashing with bcryptjs, plus session handling with express-session and cookie-parser, formed the backbone of our authentication system. These tools aligned directly with user stories about registering, logging in, and managing secure accounts.

- Browsing & Filtering Logic
- UI Design from wireframes to implementation
- Brand Development

Project Management Techniques

We orchestrated our workflow using a GitHub Projects board that detailed each stage of development. Our columns included Backlog, Ready, In Progress, In Review, and multiple Done columns specific to each sprint. At the start of each sprint, we moved user stories from the Backlog into Ready, then progressed them through In Progress and In Review. Once a feature passed peer review, it was assigned to the relevant Done column. This structure gave us a tangible overview of exactly which features had been completed in each iteration.

We typically ran roughly two-week sprints based on Agile and Dynamic Systems Development principles, and we used MS Teams for daily stand-ups rather than physical meetings. For instance, if we were nearing the end of a sprint and found ourselves behind on a feature like advanced filtering, we could see at a glance that it was stuck in In Review, discuss the blocker in our morning meeting, and shift additional team members to testing or debugging if needed. This visual approach and rapid communication loop helped us adapt whenever a story revealed unexpected complexity or when teammates required extra support.

FitXchange

Leadership and Time Management

Although we aimed to distribute tasks evenly, there were moments when certain areas like debugging or documentation needed extra attention. We drew on each member's strengths and availability to fill those gaps quickly, preventing any single feature from becoming a bottleneck. By organizing short daily syncs, we could identify blockers early and reassign tasks before they halted overall progress. This flexible approach mirrored Agile/DSDM practices, emphasizing collaboration and iterative delivery over rigid role assignments.

Ease and Challenges

Choosing front-end elements was relatively straightforward, thanks to design patterns borrowed from e-commerce-like platforms visual flows and layouts transferred naturally to FitXchange. However, ensuring robust back-end performance, especially for multi-parameter filtering, required both meticulous indexing and query optimization. From a discrete-mathematics standpoint, each filter (e.g., category, location, price range) functioned as an on/off toggle, yielding 2^n permutations for n filters. For instance, with three filters, we computed 2^3=8 possible combinations (plus sorting variants effectively leading to double the permutations $2(2^3)$ for final queries), and we used conditional logic rather than separate routes to handle them all efficiently. Meanwhile, adapting to new Node.js libraries like multer for file uploads and compression for better response times introduced a learning curve, but ultimately boosted speed, simplified future feature additions, and kept our advanced filtering queries running smoothly under varied user requests.

Leadership and Teamwork

Although we aimed to distribute tasks evenly, there were moments when certain areas especially in debugging and documentation needed extra attention. We drew on each member's strengths and availability to fill those gaps quickly, preventing any single feature from becoming a bottleneck. By organizing short daily syncs, we could identify blockers early and reassign tasks before they halted overall progress. This flexible approach mirrored Agile/DSDM practices, emphasizing collaboration and iterative delivery over rigid role assignments.

FitXchange

Reflection on Overall Learning and Skill Growth

Technical Growth:

Exploring a broad range of npm libraries such as express, mysql2, bcryptjs, openai, and multer broadened my full-stack expertise. From session management and chatbot logic to advanced data-handling strategies and file uploads, each integration reinforced the importance of structured back-end design and efficient routing. Working with these libraries also refined my understanding of how various components (controllers, models, and services) should communicate in a Node.js ecosystem.

Process and Collaboration:

Relying on Agile sprints and GitHub-based workflow management underscored the value of consistent communication and clearly defined roles. I realized that large projects succeed most reliably when every contributor has both a deep awareness of their assigned tasks and a holistic sense of the overall architecture. Pair-programming sessions, short daily syncs, and thorough code reviews ensured that each feature from advanced filtering to authentication fit neatly into the bigger picture.

Challenges and Future Focus:

Manually verifying newly uploaded outfit images turned out to be a bottleneck for any large-scale user base. A next step would be to automate this process, perhaps through computer vision reducing manual overhead while still allowing human intervention for ambiguous or complex cases. Another significant improvement would be adding automated testing (using Mocha or Jest) to catch regressions early and streamline code review. Additionally, future versions of FitXchange could benefit from blockchain-based tracking to heighten supply chain transparency, fostering responsible borrowing and returning.

Conclusion

Overall, FitXchange has sharpened my understanding of building maintainable, scalable systems under tight time constraints. The lessons I've learned such as structuring filters with discrete mathematics, leveraging containerized deployments for consistent environments, and aligning each user story with tangible sprints will be invaluable for my final year project. I plan to fold these experiences into a more advanced, automated solution that can handle complex real-world challenges with minimal friction and maximal reliability.